

EgleView Architecture

A multi-agent data platform: an Orchestrator Agent splits any uploaded file across however many business domains it touches, N real Cortex Agents own their domain's table end to end, and a second tier of delegation-only agents lets them consult each other directly.

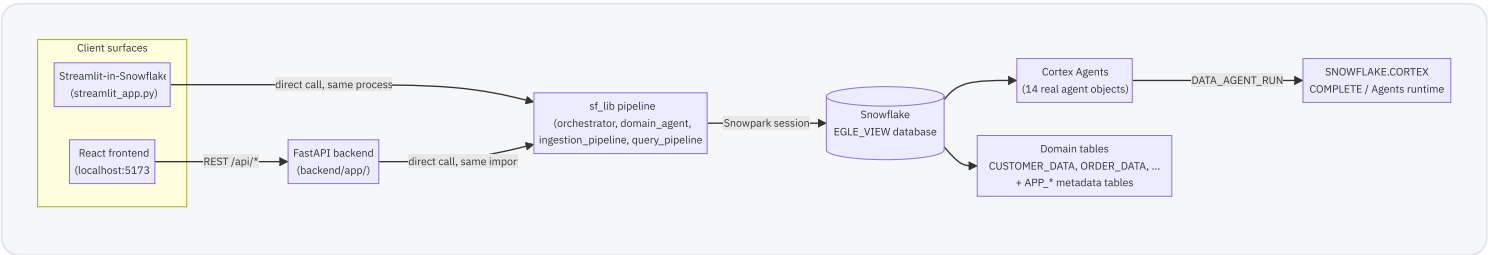
1 Overview

EgleView ingests raw business files (CSV/Excel/JSON), classifies and routes their columns to the right business domain(s), and answers natural-language questions against the resulting data — entirely through Snowflake Cortex: real tool-calling agents for every action that touches data (creating tables, writing and running SQL, asking each other questions), and plain structured LLM calls for the reasoning steps in between (routing, classification, intent understanding, final answer writing).

Two runtimes share the exact same pipeline code (`snowflake_app/sf_lib/`): a native Streamlit-in-Snowflake app, and a FastAPI + React stack. Both call the identical Orchestrator, Cortex Agents, and query pipeline — they differ only in how a Snowpark session is obtained and how the user's own schema is resolved.

2 Architecture

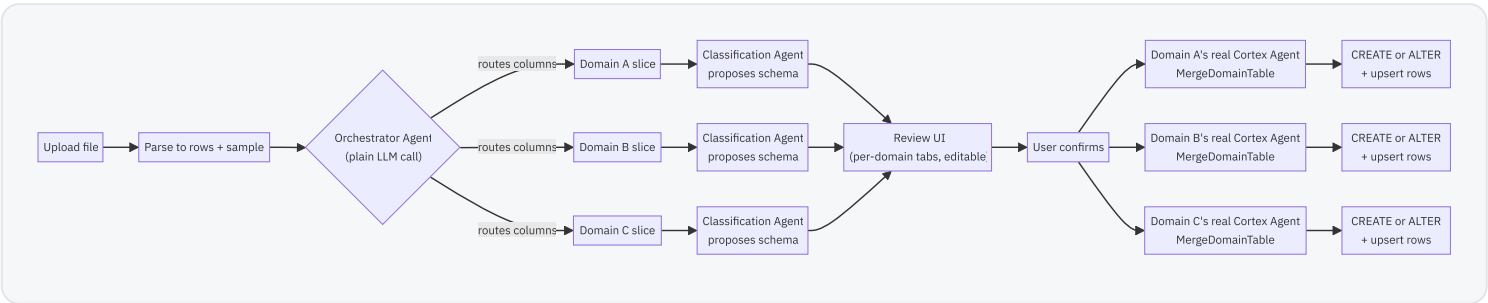
Both front ends are thin shells around one shared pipeline. Neither talks to Snowflake directly — everything funnels through `sf_lib`.



Session model differs by surface: Streamlit uses real per-user Snowflake logins (`DEMO_USER_A` / `DEMO_USER_B` , isolated by Snowflake RBAC itself); FastAPI uses one shared privileged connection and isolates users by a dynamically provisioned schema name (`USER_<hash>`) instead.

3 Ingestion workflow

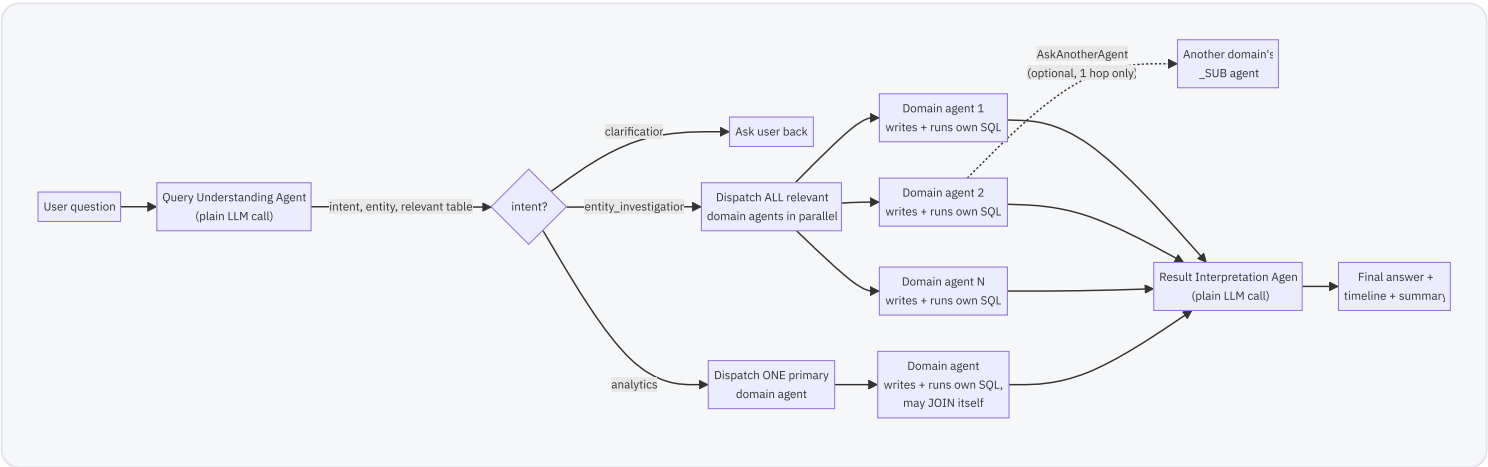
One upload can become several domains' worth of work. Everything up to "review" is proposal only — nothing touches Snowflake until the user confirms.



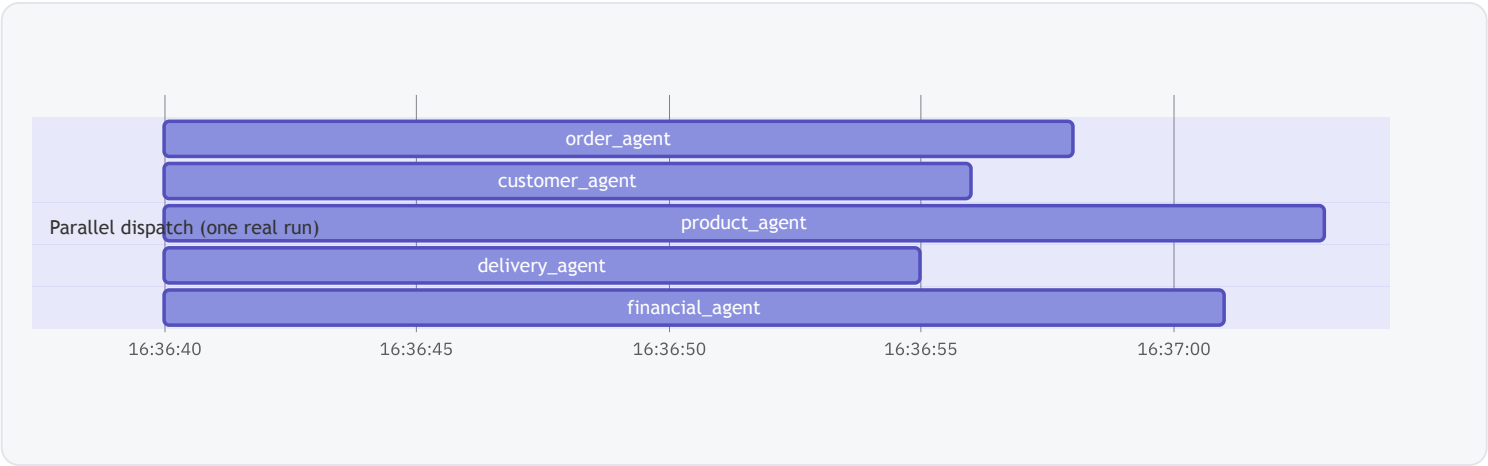
A code-level rule (not the model's judgment) force-copies every `_id`-suffixed column into every domain touched by one upload, so cross-domain joins always have a shared key later — the Orchestrator is instructed to do this itself, but wasn't reliable enough to trust alone.

4 Query workflow

SQL is authored *and* executed inside the same real agent call — there is no separate SQL-generation step. Relevant domains are dispatched in parallel, each on its own Snowflake session.



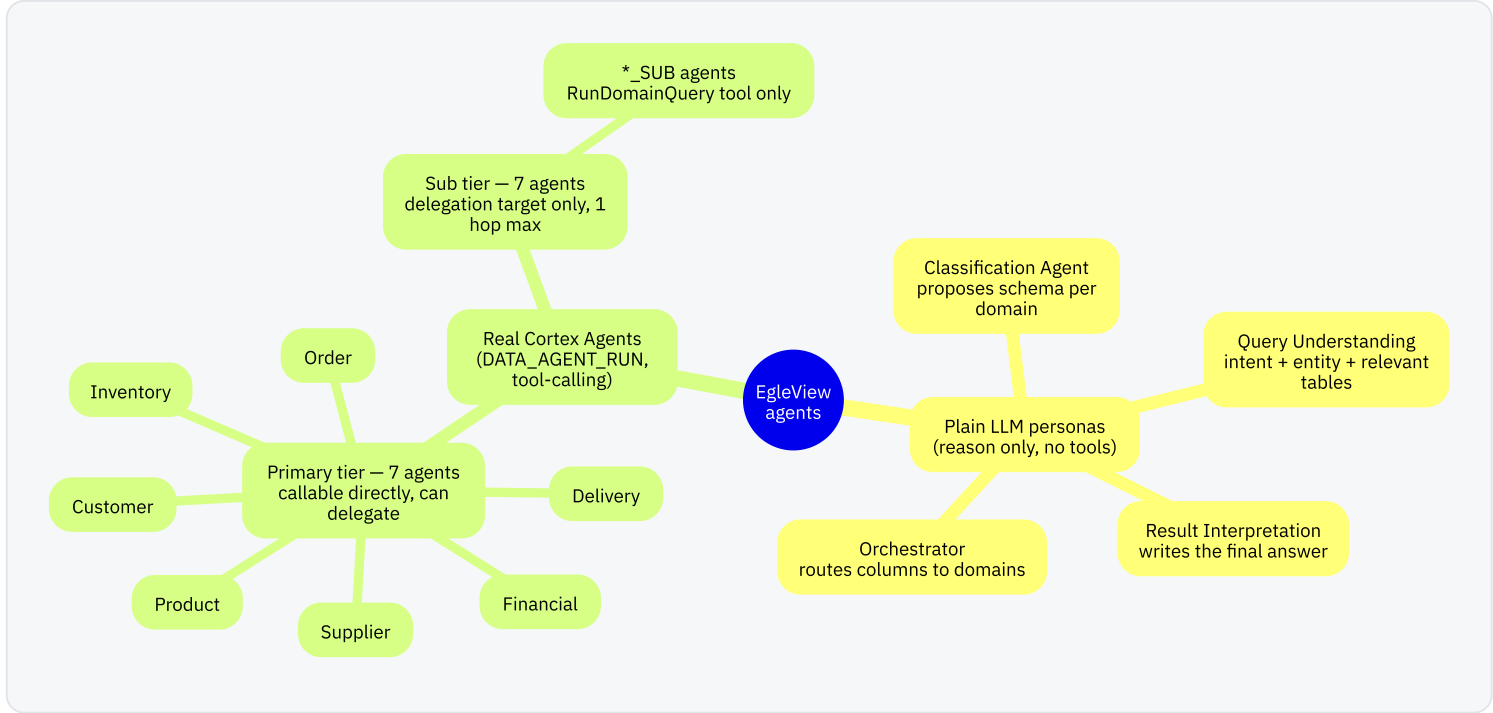
Real measured concurrency from a live test — all five domain agents dispatched at the same second, finishing at staggered times as each one's own Cortex call completes:



All five start at `16:36:40` — sequential dispatch would instead show each bar beginning only where the previous one ends.

5 Agent map

Two genuinely different kinds of "agent" make up the system — real tool-calling Cortex Agents that take actions, and plain structured-output LLM calls that only reason and decide. Conflating the two overstates what's actually agentic.



Primary agent tools

MergeDomainTable
Create-or-extend this domain's table for a confirmed schema (MERGE_TABLE_PROC)

RunDomainQuery
Write + run a read-only SELECT against its own table (RUN_SELECT_PROC)

AskAnotherAgent
Ask one other domain's `_SUB` agent a direct question (ASK_AGENT_PROC)

Recursion is prevented structurally, not by instruction: `_SUB` agents simply have no `AskAnotherAgent` tool in their spec, so a delegated question can never itself delegate — depth is capped at exactly one hop by construction.

6 Low-level design — `sf_lib` modules

The shared pipeline every runtime imports. One file, one responsibility.

MODULE	RESPONSIBILITY
<code>orchestrator.py</code>	Splits an upload's columns across domains; force-propagates <code>_id</code> columns for joinability
<code>classification.py</code>	Per-domain column mapping proposal against that domain's live existing schema
<code>domain_agent.py</code>	All real Cortex Agent calls: <code>run_agent</code> , <code>merge_table_via_agent</code> , <code>ask_about_ambiguity</code> , <code>answer_via_agent</code>
<code>ingestion_pipeline.py</code>	Stage 1 orchestration: analyze → review → confirm → merge → load
<code>query_pipeline.py</code>	Stage 2 orchestration: understand → parallel/sequential dispatch → interpret
<code>query_understanding.py</code>	Plain LLM call: intent, entity, relevant tables
<code>result_interpretation.py</code>	Plain LLM call: final answer, timeline, summary, missing info
<code>catalog_service.py</code>	Renders live domain tables + relationships as prompt text; maps table → owning agent
<code>metadata.py</code>	All CRUD against the <code>APP_*</code> Snowflake metadata tables
<code>schema_service.py</code>	Column-mapping normalization, cast-and-load dataframe build, upsert-vs-append load logic
<code>sql_validator.py</code>	Post-hoc safety check on agent-authored SQL: single SELECT/WITH, in-scope database/schema/table
<code>domain_agents.py</code>	Registry: label, concepts, fixed table name per domain
<code>activity_log.py</code>	Framework-agnostic trace: <code>st.session_state</code> in Streamlit, <code>contextvars</code> -scoped capture in FastAPI

Snowflake-side objects

MERGE_TABLE_PROC

SQL scripting · create-or-alter a domain table from a columns JSON

RUN_SELECT_PROC

Python · executes SELECT/WITH only, returns rows as JSON

ASK_AGENT_PROC

Python · resolves target's live schema, injects it, dispatches to `<KEY>_SUB`

7 Data model

Two families of tables live in the caller's own schema: fixed metadata tables (below), and one persistent table per domain with a dynamic, agent-managed schema (`CUSTOMER_DATA` , `ORDER_DATA` , ...).

TABLE	KEY COLUMNS	PURPOSE
APP_DATASETS	ID, NAME, STATUS, ROW_COUNT	One row per uploaded file
APP_DATASET_DOMAINS	ID, DATASET_ID, AGENT_KEY, TABLE_NAME, STATUS, AGENT_NOTE	One row per (upload × domain) slice
APP_DATASET_COLUMNS	DOMAIN_ID, SOURCE_COLUMN, TARGET_COLUMN, TARGET_TYPE, IS_NEW_COLUMN, INCLUDE	Proposed/edited column mapping per slice
APP_RELATIONSHIPS	SOURCE_AGENT_KEY, TARGET_AGENT_KEY, SOURCE_COLUMN, TARGET_COLUMN	Detected cross-domain join keys
APP_CONVERSATIONS	SESSION_ID, ROLE, CONTENT, SQL_GENERATED	Chat history per session

8 API contract FastAPI backend

ENDPOINT	RETURNS
POST <code>/api/datasets/upload</code>	<code>{id, name, row_count, status}</code>
POST <code>/api/datasets/seed-demo</code>	<code>{results[], agent_log[]}</code>
POST <code>/api/datasets/{id}/analyze</code>	<code>{dataset_id, domains[], agent_log[]}</code> — one entry per routed domain
POST <code>/api/datasets/{id}/confirm</code>	<code>{dataset_id, domains[], agent_log[]}</code>
GET <code>/api/datasets</code>	<code>{datasets[], domains[]}</code>
POST <code>/api/query</code>	<code>{answer, intent, entity, result, timeline, summary, agents_consulted[], agent_log[]}</code>

Every `/api/query` response — success or short-circuit (clarification, no data, not found) — carries the same full field set with empty defaults, so the client never has to guess which fields a given response included.

9 Constraints & decisions worth knowing

- **Agent message text rejects double-quotes, square/curly brackets, and literal newlines** — an undocumented `DATA_AGENT_RUN` validation quirk; every prompt built for an agent call is single-line plain prose with quotes stripped.
- **DATA_AGENT_RUN arguments must be literal SQL constants** — no bind parameters, no `PARSE_JSON(?)` wrapping a bind param.

- **Tool execution runs under a different privilege context than the calling role** — every shared proc and agent is granted to `ROLE PUBLIC` , not just the demo roles, or tool calls fail with "unknown user-defined function."
 - **A session can run one query at a time** — parallel agent dispatch opens one independent Snowflake session per concurrent call, never reuses the caller's.
 - **`contextvars.Context` isn't reentrant across threads** — each parallel worker gets its own `copy_context()` , not a shared one.
 - **SQL safety validation is post-hoc for agent-composed queries** — since authoring and execution now happen inside the same agent call, the read-only/in-scope check runs on the SQL after it already ran, not before.
-